

Elementos básicos de Python

Python es un lenguaje de programación poderoso y fácil de aprender. Cuenta con estructuras de datos eficientes y de alto nivel y un enfoque simple pero efectivo a la programación orientada a objetos. La elegante sintaxis de Python y su tipado dinámico, junto con su naturaleza interpretada, hacen de éste un lenguaje ideal para scripting y desarrollo rápido de aplicaciones en diversas áreas y sobre la mayoría de las plataformas.

1. Palabras reservadas.

Estas son palabras que **NO** se pueden emplear como nombre de variables o funciones.

and	del	for	is	raise	assert
if	else	elif	from	lambda	return
break	global	not	try	class	except
or	while	continue	exec	import	yield
def	finally	in	print		

2. Operadores.

Operadores Lógicos

Operador	Descripción	Ejemplo
and	¿se cumple a y b?	r=True and False # r es False
or	¿se cumple a o b?	r=True o False # r es True
not	No a	r=not True # r es False

AND		
A	B	Salida
False	False	False
False	True	False
True	False	False
True	True	True

OR		
A	B	Salida
False	False	False
False	True	True
True	False	True
True	True	True

NOT	
A	Salida
False	True
True	False

Operadores Relacionales

Operador	Descripción	Ejemplo
==	¿Son iguales a y b?	r=5==3 # r es False
!=	¿Son distintos a y b?	r=5!=3 # r es True
<	¿Es a menor que b?	r=5<3 # r es False
>	¿Es a mayor que b?	r=5>3 # r es True
<=	¿Es a menor o igual que b?	r=5<=5 # r es True
>=	¿Es a mayor o igual que b?	r=5>=3 # r es True

Operadores Aritméticos

Operador	Descripción	Ejemplo
+	Suma	r=3+2 # r es 5
-	Resta	r=4-7 # r es -3
-	Negación	r=-7 # r es -7
*	Multiplicación	r=2*6 # r es 12
**	Exponente	r=2**6 # r es 64
/	División	r=3.5/2 # r es 1.75
//	División Entera	r=3.5//2 # r es 1.0
%	Módulo	r=7%2 # r es 1

Adicionalmente, el símbolo + es empleado como operador de concatenación.

3. Comentarios.

Para comentar un texto (texto que no será ejecutado), se puede emplear dos símbolos: `#` o `"""` (triple comillas).

El símbolo `#`, comenta todo el texto anotado a continuación de él hasta el final de la línea.

El símbolo `"""`, comenta varias líneas hasta encontrar otro `"""` que finalice el comentario.

01	<code># Comentario de una línea.</code>
02	
04	<code>""" Este es un comentario</code>
05	<code>de varias</code>
06	<code>líneas """</code>

4. Uso Variables.

Las variables son espacios de memoria en donde es posible almacenar valores. Los tipos de datos de los valores que se pueden almacenar en una variable son:

1. Integers (Enteros) y Long
2. Floating-Point (flotante o Reales)
3. Complex Numbers (Números complejos)
4. String (Cadenas)
5. Boolean (Booleano)

Una variable se autodeclara dependiendo del valor asignado a ella por primera vez.

5. Entrada y Salida.

Para leer un valor por teclado, se emplea la función `input()`. La función `input()` entrega el valor leído como un texto.

```
variable = input()
```

La salida a pantalla se puede realizar empleando la función `print`:

```
print(valor(es)_a_desplegar)
```

La función `print` despliega en pantalla el o los valores especificados, luego el cursor queda ubicado en la siguiente línea. Para evitar esto último, se puede emplear el parámetro `end = ""`.

6. Estructuras de Control.

Una estructura de control es un bloque de código que permite agrupar instrucciones de manera controlada. En este capítulo, hablaremos sobre dos estructuras de control:

1. Estructuras de control condicionales
2. Estructuras de control iterativas

6.1. Identación

Para hablar de estructuras de control de flujo en Python, es imprescindible primero, hablar de indentación.

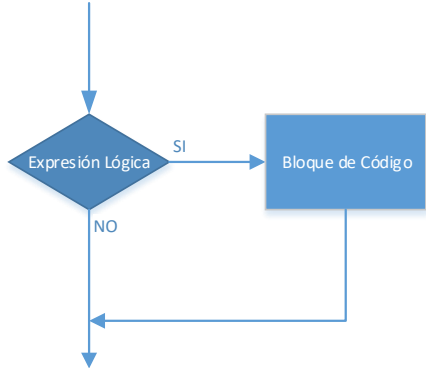
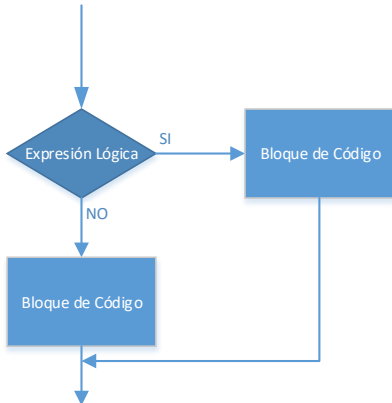
¿Qué es la indentación? En un lenguaje informático, la indentación es lo que la sangría al lenguaje humano escrito (a nivel formal). Así como para el lenguaje formal, cuando uno redacta una carta, debe respetar ciertas sangrías, los lenguajes informáticos, requieren una indentación.

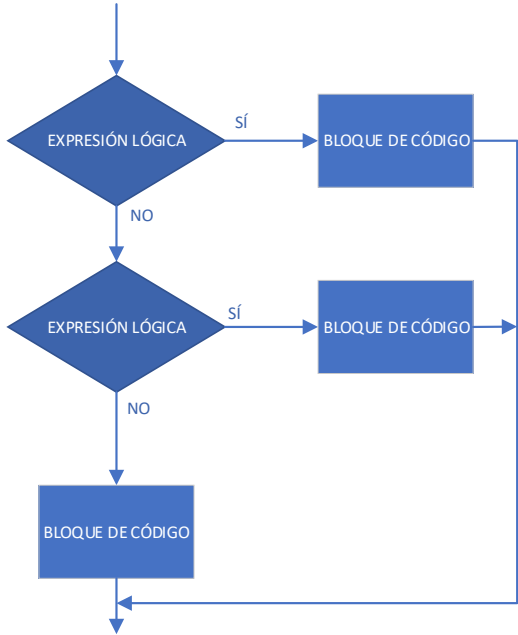
No todos los lenguajes de programación necesitan de una indentación, aunque sí, se estila implementarla, a fin de otorgar mayor legibilidad al código fuente. Pero en el caso de Python, la indentación es obligatoria, ya que, de ella dependerá su estructura.

Una estructura de control, entonces, se define de la siguiente forma:

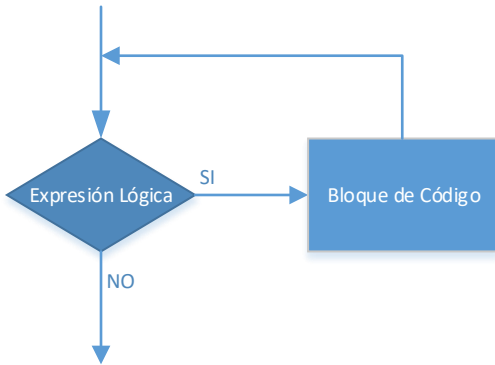
inicio de la estructura de control:
 expresiones

6.2. Estructura de Selección.

<pre>if expresión_lógica: bloque_de_código</pre>	
<pre>if expresión_lógica: bloque_de_código else: bloque_de_código</pre>	

<pre> if expresión_lógica: bloque_de_código elif expresión_lógica: bloque_de_código else: bloque_de_código </pre>	 <pre> graph TD Start(()) --> D1{EXPRESIÓN LÓGICA} D1 -- SÍ --> B1[BLOQUE DE CÓDIGO] D1 -- NO --> D2{EXPRESIÓN LÓGICA} D2 -- SÍ --> B2[BLOQUE DE CÓDIGO] D2 -- NO --> B3[BLOQUE DE CÓDIGO] B1 --> Join(()) B2 --> Join B3 --> Join Join --> End(()) </pre>
--	--

6.3. Estructuras de Iteración.

<pre> while expresión_lógica: bloque_de_código </pre>	 <pre> graph TD Start(()) --> D{Expresión Lógica} D -- SI --> B[Bloque de Código] B --> D D -- NO --> End(()) </pre>
<pre> For variable in elemento_de_control: bloque_de_código </pre>	Elemento de control: • Rango: <code>range(valor_inicio, valor_termino - 1)</code> • Tupla • Lista

7. Funciones.

Función **type**. Devuelve el tipo de dato del valor pasado como parámetro.

Función **int**. Retorna un número entero del valor pasado como parámetro.

Función **float**. Retorna un número real del valor pasado como parámetro.

Función **str**. Devuelve una cadena de caracteres (texto) del valor pasado como parámetro.

Función **random**. Entrega un valor entre 0 y 0,999999...

Función **randrange**. Entrega un valor entre 0 y n-1

Función **randint**. Entrega un valor entre n y m.